

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Experiments

Course Code:	ITA0514	Course Title:	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	<i>Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)</i>		
Student Name:	K. Vijay Bhaskar Reddy	Reg. No.:	1924251555
Section / Batch:	AI&ML/2024	Date:	10/08/2026

Code of Conduct

I K. Vijay Bhaskar Reddy(1924251555) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

K. Vijay Bhaskar Reddy

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Noise Removal Comparison

Perform a comparative study of different noise removal techniques applied to images. Explain the objectives and principles of commonly used filtering methods. Implement multiple noise reduction filters using suitable tools, document the outputs clearly, and analyze the effectiveness of each technique in preserving image details while removing noise.

Aim

To perform a comparative study of different **image noise removal techniques**, implement multiple filtering methods, and analyze their effectiveness in removing noise while preserving important image details.

Objectives

1. To understand the concept of noise in digital images.
2. To study commonly used image filtering techniques.
3. To implement different noise reduction filters.
4. To compare the quality of filtered images.
5. To determine which filter provides the best balance between **noise removal and edge/detail preservation**.

Principle

Image noise is an unwanted random variation in pixel intensity. It can occur because of poor lighting, sensor limitations, transmission errors, or image acquisition problems.

Common noise-removal techniques work by modifying a pixel based on its neighboring pixels.

Common filters

Filter	Principle	Best suited for	Main limitation
Mean Filter	Replaces pixel with average of neighborhood	Gaussian/random noise	Blurs edges
Median Filter	Replaces pixel with median of neighborhood	Salt-and-pepper noise	Can remove fine details
Gaussian Filter	Weighted average using Gaussian distribution	Gaussian noise	Some edge blurring
Bilateral Filter	Considers spatial and intensity differences	Noise with edge preservation	Computationally slower

Non-Local Means	Averages similar patches from different image locations	General/grain noise	More computationally expensive
------------------------	---	---------------------	--------------------------------

Python Implementation

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("input.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
noise = np.random.normal(0, 25, img.shape)
noisy = np.clip(img + noise, 0, 255).astype(np.uint8)
mean = cv2.blur(noisy, (5, 5))
median = cv2.medianBlur(noisy, 5)
gaussian = cv2.GaussianBlur(noisy, (5, 5), 0)
bilateral = cv2.bilateralFilter(noisy, 9, 75, 75)
nlm = cv2.fastNlMeansDenoisingColored(
    noisy, None, 10, 10, 7, 21
)
images = [img, noisy, mean, median, gaussian, bilateral, nlm]
titles = [
    "Original",
    "Noisy Image",
    "Mean Filter",
    "Median Filter",
    "Gaussian Filter",
    "Bilateral Filter",
    "Non-Local Means"
]

plt.figure(figsize=(14, 8))

for i in range(len(images)):
    plt.subplot(2, 4, i + 1)
    plt.imshow(images[i])
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

Original



Noisy Image



Gaussian Filter



Bilateral Filter



Mean Filter



Median Filter



Non-Local Means



Procedure

Step 1: Input Image

Take a clean image as the input.

Step 2: Add Noise

Artificially add Gaussian noise to the image to create a noisy image.

Step 3: Apply Filters

Apply the following filters:

1. Mean filter
2. Median filter
3. Gaussian filter
4. Bilateral filter
5. Non-Local Means filter

Step 4: Display Results

Display the original, noisy, and filtered images side by side.

Step 5: Compare

Compare each output based on:

- Noise reduction
- Edge preservation
- Fine-detail preservation
- Image smoothness
- Computational complexity

Technique	Noise Removal	Edge Preservation	Detail Preservation
Mean	Good	Poor	Poor
Median	Very good for salt-and-pepper	Good	Good
Gaussian	Good	Moderate	Moderate
Bilateral	Very good	Excellent	Excellent
Non-Local Means	Excellent	Very good	Very good

Expected Observation Analysis

Mean Filter:

The mean filter effectively smooths noise but also averages important edge pixels. Therefore, the resulting image may appear blurry.

Median Filter:

The median filter is particularly effective against **salt-and-pepper noise**. It removes isolated bright and dark pixels while preserving edges better than the mean filter.

Gaussian Filter:

Gaussian filtering produces smooth results and is effective for Gaussian noise. However, it can soften edges and fine structures.

Bilateral Filter:

The bilateral filter performs spatial smoothing while considering pixel intensity differences. Therefore, it can reduce noise while maintaining sharp edges.

Non-Local Means:

Non-Local Means searches for similar image patches and averages them. It generally provides strong noise reduction while retaining considerable image detail, but requires more computation.

Result

The different noise-removal techniques were successfully implemented and compared. The **Mean, Median, Gaussian, Bilateral, and Non-Local Means filters** successfully reduced image noise with different levels of detail preservation.

Among the tested methods, **Bilateral Filtering** provides an excellent balance between noise reduction and edge preservation, while **Non-Local Means** can provide superior denoising for suitable images at a higher computational cost.

Conclusion

The experiment demonstrates that **no single filter is optimal for every type of noise**. Median filtering is preferable for salt-and-pepper noise, Gaussian filtering is useful for Gaussian noise, while Bilateral and Non-Local Means filtering are better choices when **preserving image edges and fine details** is important.